

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
Кафедра «Информационная безопасность в вычислительных системах и сетях»

МЕТОДИЧЕСКИЕ УКАЗАНИЯ
к практическому занятию № 1

«Спецификация алгоритма и анализ псевдокода»
по дисциплине «Алгоритмы и структуры данных»

Параметр	Значение
Направление подготовки	03.03.01 Прикладные математика и физика
Направленность	03.03.01 Прикладные физика и математика
Семестр	7
Продолжительность	2 академических часа
Компетенция / индикатор	ПК-4 / ПК-4.4
Статус	Проект методических указаний

Ростов-на-Дону
7 сентября 2026 г.

1. Паспорт практического занятия

Основание разработки. Актуализированная рабочая программа дисциплины «Алгоритмы и структуры данных» для направления 03.03.01, раздел 1 «Основы алгоритмизации и анализа сложности», 32 часа практических занятий (16 работ).

Место в курсе. Первая работа формирует единый язык описания алгоритмов перед изучением структур данных и экспериментальной оценки трудоёмкости. Практическая работа № 2 продолжит тему доказательством корректности циклов и построением воспроизводимой серии измерений времени и памяти.

Цель. Научиться переводить содержательную вычислительную задачу в однозначную спецификацию, читать и трассировать псевдокод, формулировать инвариант цикла и получать оценки времени и дополнительной памяти.

1.1. Планируемые результаты

По завершении работы обучающийся:

- формулирует предусловие и постусловие алгоритма, различая входные данные, результат и ограничения;
- выполняет пошаговую трассировку псевдокода на нормальном и граничном наборах данных;
- выделяет основную операцию и получает оценку трудоёмкости в обозначениях O , Ω и Θ ;
- формулирует инвариант цикла и использует его для краткого обоснования корректности;
- реализует алгоритм на Python 3.12+ и подтверждает результат набором воспроизводимых тестов.

1.2. Связь с РПД и оцениванием

Элемент	Реализация в работе	Подтверждение
ПК-4 / ПК-4.4	Использование современного программного средства для решения прикладной задачи	Исходный код и тестовый запуск
Базовый результат	Распознавание операций и оценка простого фрагмента	Трассировка и асимптотическая оценка
Средний результат (элемент)	Обоснование выбора линейного прохода	Сравнение с наивной альтернативой
Текущий контроль	Практическая работа № 1	Отчёт, код, ответы при защите

2. Входная подготовка и ресурсы

До занятия необходимо повторить переменные, условный оператор, цикл `for`, индексацию последовательностей, функции и базовые логические выражения. Нужно уметь запускать программу и интерпретировать сообщение об ошибке.

Программная среда:

- обязательный вариант — Python 3.12 или новее;
- среда — командная строка, JupyterLab, VSCodium или Visual Studio Code;
- допустимая сравнительная реализация — C++20 с компилятором GCC/G++;
- для сдачи сохраняются исходный код, входные данные (если они вынесены в файл) и краткий отчёт.

Безопасность и организация работы. Используйте только учебные или обезличенные наборы данных. Не запускайте неизвестный код с сетевым или файловым доступом. Соблюдайте требования компьютерного класса и делайте краткие перерывы при длительной работе за монитором.

3. Краткие теоретические сведения

3.1. Спецификация алгоритма

Спецификация отвечает на вопрос «что должно быть вычислено», не навязывая способ решения. В данной работе используется форма логического контракта $\{P\} A \{Q\}$, где P – предусловие, A – алгоритм, Q – постусловие.

Компонент	Контрольный вопрос	Типичная ошибка
Вход	Какие объекты и типы получает алгоритм?	Не указан тип или допустимый диапазон
Предусловие P	Когда запуск алгоритма корректен?	Скрыто предполагается непустой массив
Постусловие Q	Что именно истинно после завершения?	Записан процесс вместо свойства результата
Граничные случаи	Что происходит при минимальном размере и равных значениях?	Не определено правило выбора при равенстве

3.2. Корректность и инвариант

Инвариант цикла – утверждение, истинное перед первой итерацией, сохраняющееся после каждой итерации и вместе с условием завершения приводящее к постусловию. Краткое доказательство включает три шага: инициализацию, сохранение и завершение.

3.3. Трудоёмкость

Для входа размера n выбирается основная операция, после чего оценивается число её выполнений. $O(g(n))$ задаёт асимптотическую верхнюю границу, $\Omega(g(n))$ – нижнюю, $\Theta(g(n))$ – точный порядок роста с точностью до постоянного множителя. Дополнительная память учитывает объекты, создаваемые алгоритмом сверх хранения входа.

Важно. Если цикл всегда просматривает все n элементов, трудоёмкость может быть $\Theta(n)$ и в лучшем, и в худшем случае. Значения данных влияют на число успешных ветвлений, но не обязательно меняют порядок роста.

4. Порядок выполнения

1. Получите вариант задания и выпишите вход, выход и ограничения.
2. Запишите контракт $\{P\} A \{Q\}$; отдельно определите поведение на граничных данных.
3. Составьте псевдокод, используя присваивание, ветвление и цикл без привязки к синтаксису языка.
4. Выполните ручную трассировку минимум на трёх наборах: обычном, минимальном и содержащем равные/граничные значения.
5. Сформулируйте инвариант основного цикла и кратко проверьте инициализацию, сохранение и завершение.
6. Выберите основную операцию, подсчитайте число итераций и получите O -, Ω - и Θ -оценки времени и памяти.
7. Реализуйте алгоритм на Python; добавьте не менее пяти автоматизированных проверок `assert`.
8. Сопоставьте полученный результат с контрактом, оформите вывод и подготовьте файлы к сдаче.

5. Демо-пример

5.1. Постановка задачи

Дана непустая последовательность конечных вещественных измерений $x[0 \dots n - 1]$. Найти максимальное по модулю измерение и индекс его первого появления. Такое правило позволяет выявить наибольшее отклонение независимо от знака и однозначно разрешает совпадения.

Вход: массив x длины $n \geq 1$, каждый элемент — конечное вещественное число.

Выход: пара $(k, x[k])$, где $|x[k]| = \max |x[i]|, 0 \leq i < n$; если максимум достигается несколько раз, выбирается минимальный индекс k .

Контракт: $P \equiv (n \geq 1) \wedge (\forall i: x[i] \text{ — конечное число}); Q \equiv (0 \leq k < n) \wedge (\forall i: |x[k]| \geq |x[i]|) \wedge (\forall j < k: |x[j]| < |x[k]|)$.

5.2. Псевдокод

```
MAX_ABS_FIRST(x)
  n ← length(x)
  k ← 0
  for i ← 1 to n - 1 do
    if abs(x[i]) > abs(x[k]) then
      k ← i
  return (k, x[k])
```

Строгое сравнение «>» сохраняет индекс первого элемента при равенстве модулей. Замена на «≥» изменила бы контракт: выбиралось бы последнее появление.

5.3. Трассировка

Для $x = [-1,5; 2,0; -3,2; 3,2; 0,4]$ ожидается $k = 2$ и $x[k] = -3,2$.

i	x[i]	x[i]	k до	Сравнение	k после
1	2,0	2,0	0	$2,0 > 1,5$ — да	1
2	-3,2	3,2	1	$3,2 > 2,0$ — да	2
3	3,2	3,2	2	$3,2 > 3,2$ — нет	2
4	0,4	0,4	2	$0,4 > 3,2$ — нет	2

5.4. Обоснование корректности

Инвариант. Перед итерацией с индексом i переменная k хранит минимальный индекс элемента с максимальным модулем среди $x[0 \dots i - 1]$.

Инициализация. Перед первой итерацией $i = 1$ просмотрен только $x[0]$, поэтому $k = 0$ удовлетворяет инварианту.

Сохранение. Если $|x[i]|$ больше текущего максимума, присваивание $k \leftarrow i$ делает новый элемент единственным лучшим. Иначе прежний k остаётся первым индексом максимума; при равенстве он не меняется.

Завершение. После последней итерации просмотрен весь массив $x[0 \dots n - 1]$, поэтому инвариант совпадает с постусловием Q.

5.5. Оценка ресурсов

Цикл выполняет $n - 1$ сравнение модулей для любого допустимого входа. Следовательно, время работы $T(n) = \Theta(n)$; одновременно $T(n) = O(n)$ и $T(n) = \Omega(n)$. Алгоритм хранит несколько скалярных переменных, поэтому дополнительная память $M(n) = \Theta(1)$.

Почему линейный просмотр обоснован. Чтобы гарантировать максимум в неупорядоченной последовательности, в общем случае необходимо учесть каждый элемент: непрочитанный элемент мог бы иметь больший модуль. Поэтому нижняя граница $\Omega(n)$ совпадает с достигнутой верхней границей.

5.6. Реализация и тесты

```
from math import isfinite

def max_abs_first(x: list[float]) -> tuple[int, float]:
    if not x:
        raise ValueError("sequence must be non-empty")
    if not all(isfinite(v) for v in x):
        raise ValueError("all values must be finite")

    k = 0
    for i in range(1, len(x)):
        if abs(x[i]) > abs(x[k]):
            k = i
    return k, x[k]

assert max_abs_first([-1.5, 2.0, -3.2, 3.2, 0.4]) == (2, -3.2)
assert max_abs_first([7.0]) == (0, 7.0)
assert max_abs_first([-2.0, 2.0]) == (0, -2.0)
```

```
assert max_abs_first([0.0, 0.0, 0.0]) == (0, 0.0)
assert max_abs_first([1.0, -5.0, 4.9]) == (1, -5.0)
```

Ожидаемый результат запуска: программа завершается без сообщений; это означает, что все инструкции `assert` истинны. Для проверки нарушения предусловия отдельно вызовите функцию с пустым списком и убедитесь, что возникает `ValueError`.

6. Индивидуальные домашние задания

Для назначенного варианта необходимо выполнить единый комплект действий: (1) контракт $\{P\} A \{Q\}$; (2) псевдокод; (3) три трассировки; (4) инвариант и краткое доказательство; (5) O -, Ω -, Θ -оценки времени и оценку дополнительной памяти; (6) реализацию Python; (7) не менее пяти `assert`-тестов. Если условие допускает несколько решений, выбранное правило следует записать явно.

№	Название	Условие
1	Минимум и последнее появление	Найти минимальное значение массива и наибольший индекс, на котором оно встречается.
2	Первое превышение порога	Найти первый индекс i , для которого $x[i] > \tau$; если такого элемента нет, вернуть -1 .
3	Ближайшее к эталону	Найти первое измерение, минимизирующее $ x[i] - a $ для заданного эталона a .
4	Максимальный скачок	Найти первый индекс $i > 0$ с максимальным значением $ x[i] - x[i-1] $.
5	Число смен знака	Подсчитать переходы между строго положительными и строго отрицательными соседними значениями; нули не считать сменой.
6	Самая длинная серия допустимых значений	Найти длину и начало первой максимальной серии элементов из диапазона $[L, U]$.
7	Первый локальный максимум	Найти первый внутренний элемент $x[i]$, который строго больше обоих соседей; иначе вернуть -1 .
8	Контроль монотонности	Определить, является ли последовательность неубывающей; при нарушении вернуть индекс первой нарушающей пары.
9	Максимум относительного отклонения	При ненулевом эталоне a найти первый индекс максимума $ x[i] - a / a $.
10	Количество выбросов	Подсчитать элементы, для которых $ x[i] - m > d$, где m и d заданы, $d \geq 0$.
11	Самая длинная серия одинаковых кодов	Для массива целых кодов найти начало и длину первой самой длинной серии равных соседних значений.
12	Минимальный интервал между событиями	Для строго возрастающего массива времён найти первую пару соседних событий с минимальным интервалом.

7. Требования к отчёту и воспроизводимости

Отчёт представляется в DOCX или PDF, исходный код – отдельным файлом .py. Рекомендуемое имя комплекта: PR01_Фамилия_ВариантNN. Отчёт должен содержать:

- титульные сведения: дисциплина, номер и тема работы, ФИО, группа, вариант, дата;
- постановку задачи и контракт $\{P\} \wedge \{Q\}$;
- псевдокод и обозначения;
- таблицы трассировки для трёх тестов;
- инвариант и три части обоснования корректности;
- оценку времени и дополнительной памяти с пояснением;
- исходный код или ссылку на приложенный файл, перечень тестов и фактический результат запуска;
- краткий вывод: что проверено, какие ограничения остаются.

Воспроизводимость. Укажите версию Python и команду запуска. Все исходные данные, необходимые для повторения результата, должны входить в комплект. Если использованы внешние фрагменты кода, библиотеки или генеративный ИИ, укажите это и опишите, что именно проверено автором работы. Передача персональных или закрытых данных внешним сервисам не допускается.

8. Критерии оценивания

Компонент	Баллы	Критерий полного выполнения
Постановка и контракт	15	Вход, выход, ограничения, P и Q заданы однозначно
Псевдокод	15	Алгоритм конечен, детерминирован и соответствует контракту
Трассировка	15	Есть три содержательных теста и корректные промежуточные состояния
Корректность	20	Сформулирован инвариант; разобраны инициализация, сохранение, завершение
Анализ ресурсов	15	Обоснованы O, Ω, Θ времени и дополнительная память
Реализация и тесты	15	Код запускается; не менее пяти проверок, включая границы
Отчёт и вывод	5	Структура соблюдена, результат воспроизводим, ограничения названы

Шкала: 91–100 баллов – «отлично»; 76–90 – «хорошо»; 61–75 – «удовлетворительно»; менее 61 – работа требует доработки. Критическая ошибка в контракте или алгоритм, не завершающийся на допустимом входе, не позволяет считать работу выполненной до исправления.

9. Вопросы для самопроверки

1. Чем спецификация алгоритма отличается от его реализации?
2. Что должно входить в предусловие и почему нельзя скрывать требование непустого массива?
3. Как сформулировать постусловие так, чтобы оно проверяло результат, а не пересказывало шаги?
4. Как строгое и нестрогое сравнения меняют правило выбора при равенстве значений?
5. Какие три части включает доказательство корректности с помощью инварианта цикла?
6. Почему инвариант должен быть истинным до первой итерации?
7. В чём различие обозначений O , Ω и Θ ?
8. Может ли алгоритм иметь $\Theta(n)$ в лучшем и худшем случаях? Приведите объяснение.
9. Что относится к дополнительной памяти алгоритма?
10. Почему одного успешного теста недостаточно для доказательства корректности?
11. Какие граничные случаи обязательны для задачи поиска экстремума?
12. Как проверить, что правило выбора первого или последнего совпадения реализовано верно?
13. Почему линейный просмотр является асимптотически оптимальным для поиска максимума в неупорядоченном массиве?
14. Какие сведения необходимо сохранить, чтобы другой человек воспроизвёл запуск программы?
15. Как корректно раскрыть использование внешнего кода или генеративного ИИ в учебной работе?

10. Методические рекомендации преподавателю

Этап	Время	Деятельность
Входная диагностика	10 мин	Коротко проверить понимание входа, выхода, цикла и индексации
Разбор контракта	15 мин	Сопоставить точную и неоднозначную формулировки задачи
Совместный демо-пример	25 мин	Провести трассировку и вывести инвариант вопросами к группе
Индивидуальная работа	30 мин	Студенты оформляют контракт, псевдокод и тесты своего варианта
Взаимопроверка	5 мин	Обменяться контрактами и найти скрытое предположение
Рефлексия и выдача ДЗ	5 мин	Зафиксировать типовые ошибки и требования к сдаче

Рекомендуется не выдавать студентам готовую реализацию индивидуальных вариантов. При защите следует изменять один граничный параметр или тест, чтобы проверить понимание контракта и устойчивость решения.

11. Основные источники

1. Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. Introduction to Algorithms. 4th ed. MIT Press, 2022.
2. Skiena S. S. The Algorithm Design Manual. 3rd ed. Springer, 2020.
3. Morin P. Open Data Structures. Open textbook. Актуальная электронная редакция.
4. Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/> (дата обращения: 07.09.2026).

12. Лист самоконтроля перед сдачей

- ☐ Контракт определяет допустимый вход и проверяемое свойство результата.
- ☐ Правило обработки равных значений записано явно.
- ☐ Псевдокод и программа реализуют одно и то же правило.
- ☐ Трассировки включают обычный, минимальный и граничный тесты.
- ☐ Инвариант согласован с границами цикла.
- ☐ Оценки времени и памяти снабжены объяснением.
- ☐ Код запускается в указанной версии Python, все тесты проходят.
- ☐ В отчёте нет персональных данных и нераскрытых внешних заимствований.